



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Sessi on
- 2

Cour se
I nst r uct or
Pr of. V R Badr i

Microprocessor & Computer Architecture (μpCA)



UE24CS251B

Processor design
trade-offs

What does the processor do?

Misconception - computers spend their time computing, i.e., carrying out arithmetic operations on user data.

- Although they do a fair amount of arithmetic, most of this is with addresses in order to locate the relevant data items and program routines.
 - Then, having found the user's data, most of the work is in moving it around rather than processing it in any transformational sense.

Consider a Scenario : The programmer generally wants to express his or her program in as abstract a way as possible, using a high-level language which supports ways of handling concepts that are appropriate to the problem

- Modern trends towards functional and object-oriented languages move the level of abstraction higher.
 - The **semantic gap** between a high-level language construct and a machine instruction is

- A typical set of statistics is shown below were gathered running a print preview program on an ARM instruction emulator, but are broadly typical of what may be expected from other programs and instruction sets.

Instruction type	Dynamic usage
Data movement	43%
Control flow	23%
Arithmetic operations	15%
Comparisons	13%
Logical operations	5%
Other	1%

- Instructions to optimize are those concerned with data movement, either between the processor registers and memory or from register to register.
- Secondly, the control flow instructions such as branches and procedure calls.
- Arithmetic operations are down at 15% as are comparisons at 13%

Now, that it is clear what processors spend their time doing, is there a way to make the processor work faster ?

- The most important of these is pipelining.
 - A very effective way of exploiting concurrency in a general-purpose processor.
- Another important technique is the use of a cache memory.
 - A CPU cache is a cache used by the CPU to reduce the average time to access data from the main memory.
- A third technique, super-scalar instruction execution.
 - A superscalar processor executes more than one instruction during a clock cycle by simultaneously dispatching multiple instructions to different execution units on the processor.



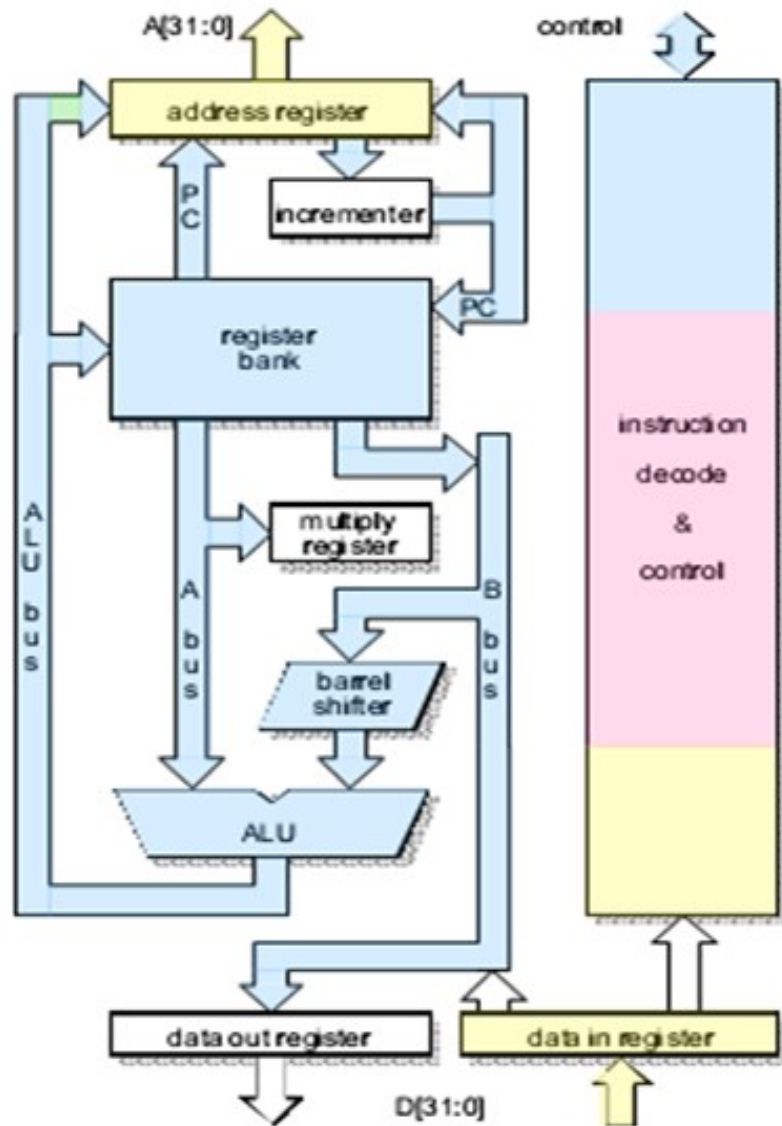
Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

ARM ISA:
Instruction Set
Architecture

ARM Processor – 3 stage Pipeline

Architecture



The ARM [ACORN RISC MACHINE] is a RISC processor

3 stages of Pipeline

fetch

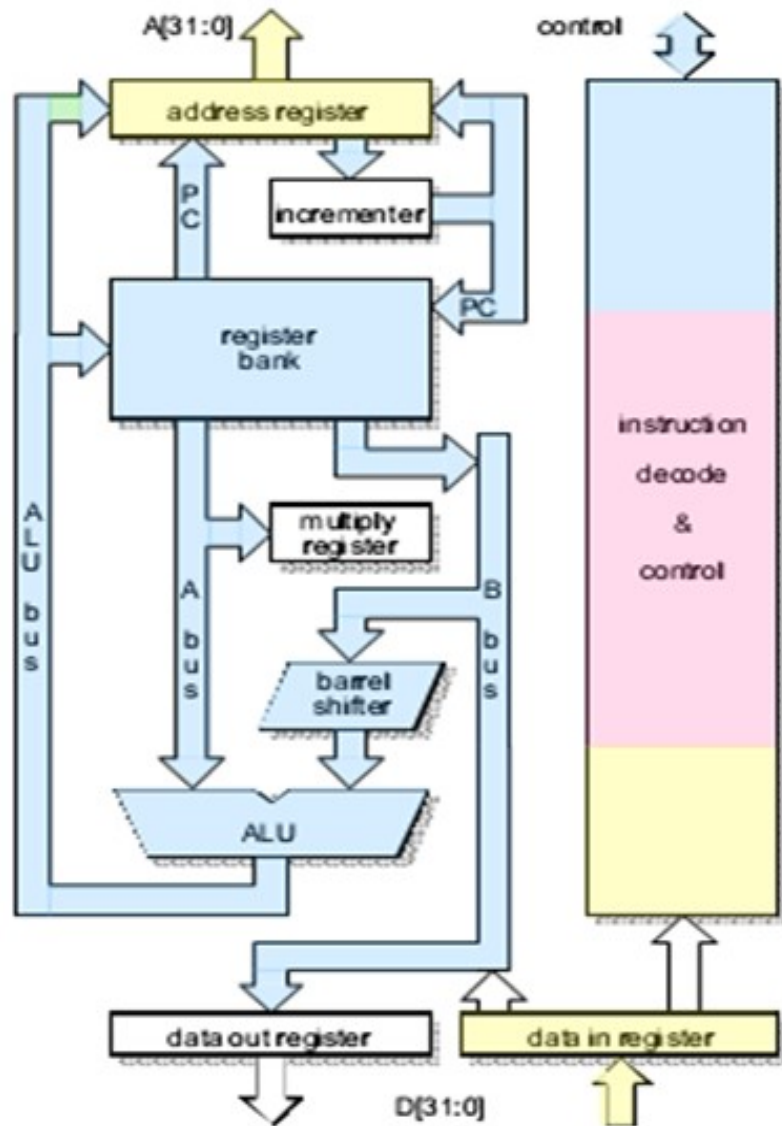
decode

execute

• The RISC features incorporated by ARM

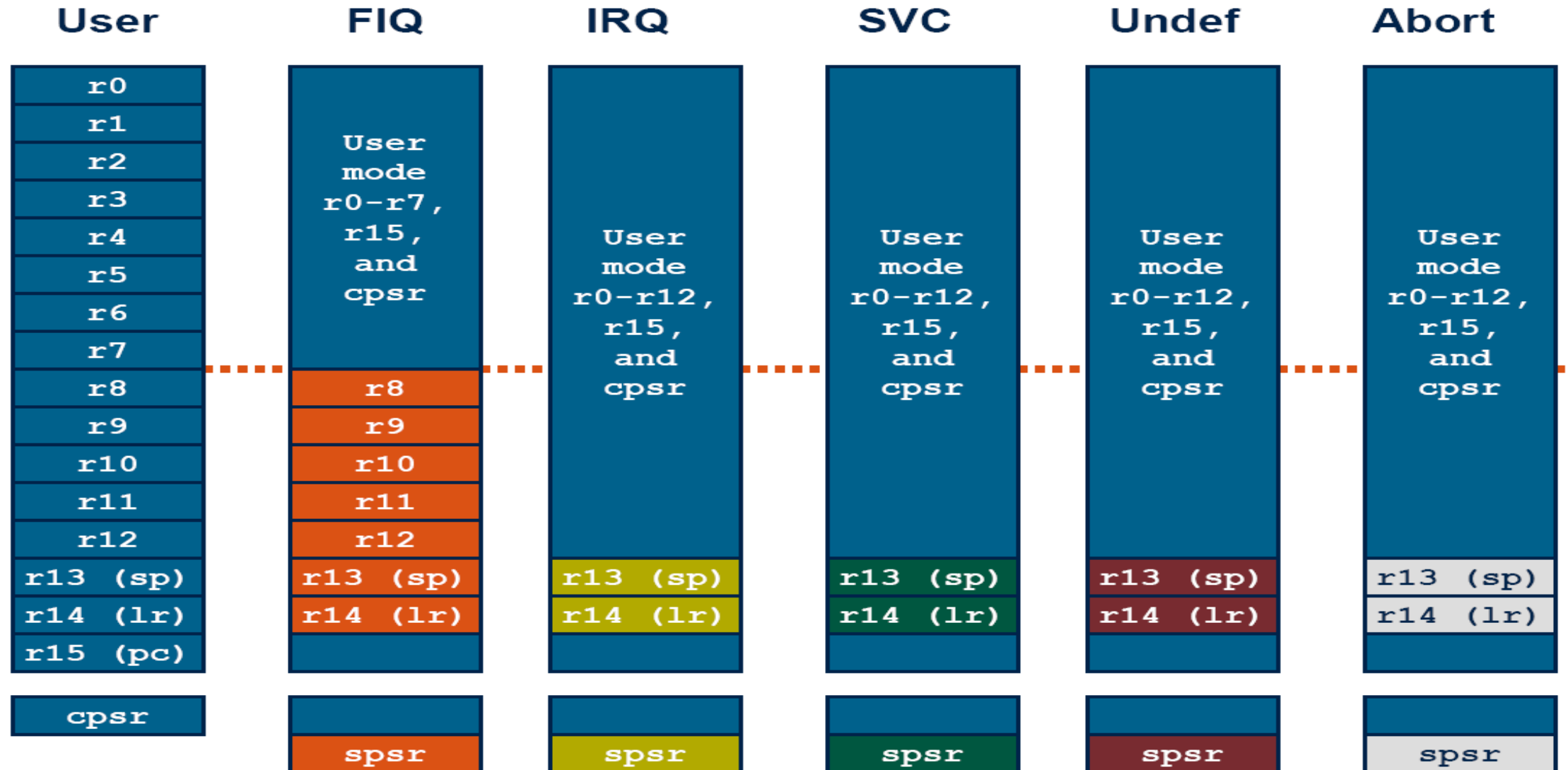
- A load-store architecture.
- Fixed-length 32-bit instructions.
- 3-address instruction formats

Organization of ARM – 3 stage PP



- The **register bank**, which stores the processor state.
 - Two read ports and One write port - to access any register
 - Plus an **additional read port** and an **additional write port** that give special access to r15
- The **barrel shifter**, which can **shift** or **rotate** one operand by any number of bits.
- The **ALU**, which performs the **arithmetic and logic** functions required by the instruction set.
- The **address register** and **incrementer**, which select and hold all memory addresses and **generate sequential addresses** when required.

Register organization & Modes



The ARM Register Set – SVC Mode

Current Visible Registers

SVC Mode

r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)
cpsr
spsr

Banked out Registers

User	FIQ	IRQ	Undef	Abort
	r8			
	r9			
	r10			
	r11			
	r12			
r13 (sp)	r13 (sp)	r13 (sp)	r13 (sp)	r13 (sp)
r14 (lr)	r14 (lr)	r14 (lr)	r14 (lr)	r14 (lr)
	spsr	spsr	spsr	spsr

The ARM Register Set – Undefined Mode

Current Visible Registers

Undef Mode

r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)
cpsr
spsr

Banked out Registers

User

FIQ

IRQ

SVC

Abort

r13 (sp)
r14 (lr)

r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)

r13 (sp)
r14 (lr)

r13 (sp)
r14 (lr)

r13 (sp)
r14 (lr)

spsr

spsr

spsr

spsr

The ARM Register Set – IRQ Mode

Current Visible Registers

IRQ Mode

r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)
cpsr
spsr

User

r13 (sp)
r14 (lr)

FIQ

r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
spsr

Banked out Registers

SVC

r13 (sp)
r14 (lr)
spsr

Undef

r13 (sp)
r14 (lr)
spsr

Abort

r13 (sp)
r14 (lr)
spsr

The ARM Register Set – FIQ Mode

Current Visible Registers

FIQ Mode

r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)
cpsr
spsr

User

r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)

Banked out Registers

IRQ

SVC

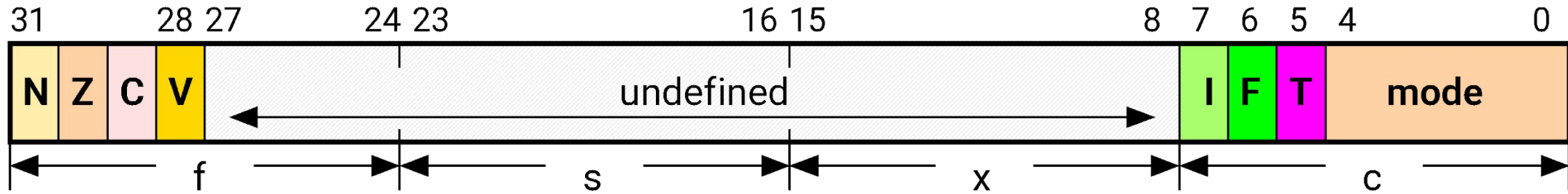
Undef

Abort

r13 (sp)	r13 (sp)	r13 (sp)	r13 (sp)
r14 (lr)	r14 (lr)	r14 (lr)	r14 (lr)

spsr	spsr	spsr	spsr
------	------	------	------

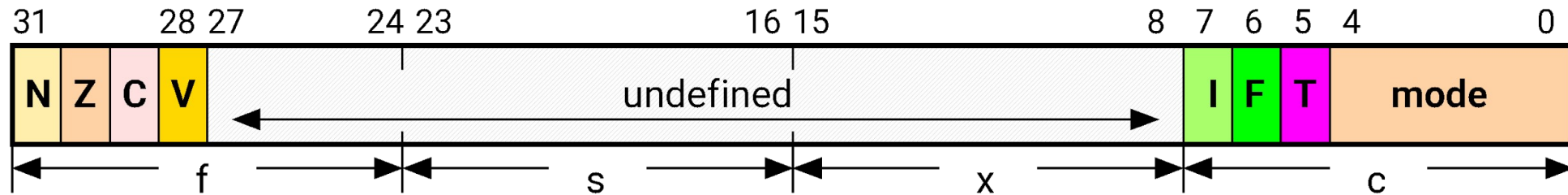
Status Register (SR)



SR fields are divided into four fields, each 8-bit wide:

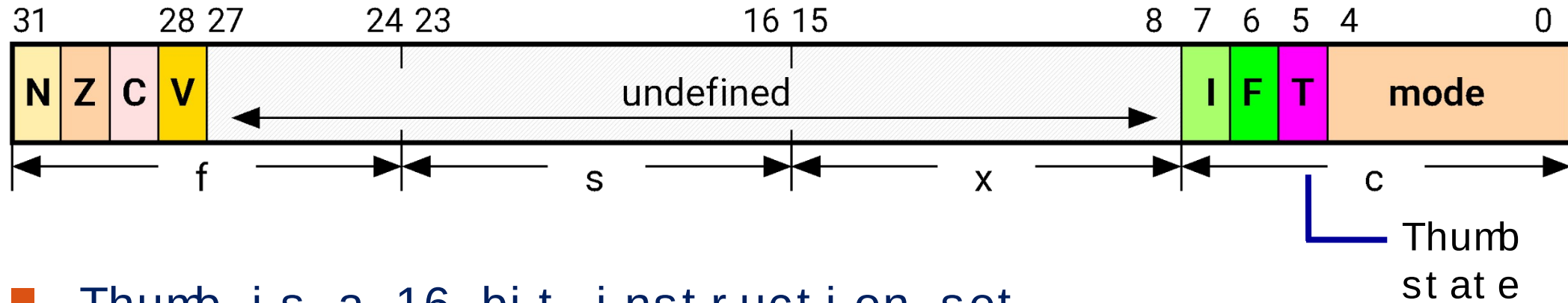
- Flags (f)
- Status (s) : *Reserved for Future*
- Extension (x) : *Reserved for Future*
- Control (c).

Status Register (SR)



- **N: Negative:** Set to 1 if the Result of previous instruction is Negative.
 - **Z: Zero:** Set to 1 if the Result of previous instruction is Zero.
 - **C: Carry:** Set to 1, if result of either an Arithmetic or Shifter produce a carry-out, **Mode bits**
 10000 User
 10001 FIQ
 10010 IRQ
 10011 Supervisor
 10111 Abort
 11011 Undefined
 11111 System
 - **V: overflow:** Set to 1, if previous instruction produce an overflow into the sign bit.
- Interrupt Disable bits.**
 I = 1: Disables the IRQ.
 F = 1: Disables the FIQ.
- T Bit (Arch. with Thumb mode only)**
 T = 0: Processor in ARM state
 T = 1: Processor in Thumb state

State Register (SR)



- Thumb is a 16-bit instruction set
 - Optimized for code density from C code (~65% of ARM code size)
 - Improved performance from memory with a narrow data bus
 - Subset of the functionality of the ARM instruction set
 - Only Low Registers R0- R7 are used
 - Constants are of limited size.

Status Register : CPSR & SPSR

Current Program Status Register: CPSR

- ARM core uses CPSR to monitor & control internal operations.
- The current status of the program under execution, such as, result of current execution instruction is zero/ -ve are captured here.

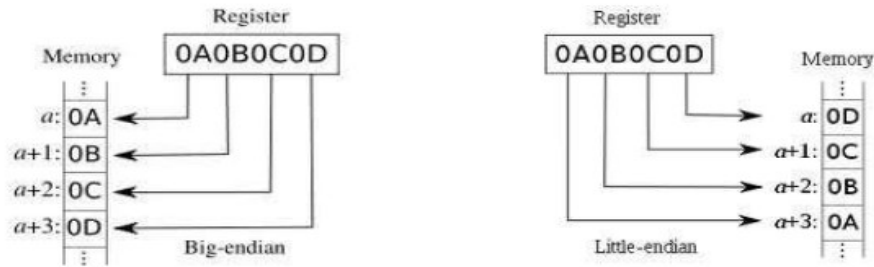
Saved Program Status Register: SPSR

- Processor, While Shifting one mode to another mode, CPSR will be copied to SPSR.
- SPSR will be copied back to CPSR when it return back to previous mode.

Example:

Suppose Processor is in USER mode of operation and if IRQ request arrives then processor has to switch itself to IRQ mode of operation. First CPSR is get copied to SPSR then processor will serve IRQ mode. After serving IRQ mode processor should return to USER mode and should resume its working. So SPSR is get copied again into CPSR to serve USER mode.

Big Endian vs. Little Endian



- ARM processor supports both **little endian & big endian** methods for storing data in memory.

ARM processor s support six data types.

- 8-bit** signed and unsigned bytes.
- 16-bit** signed and unsigned half-words; these are aligned on 2-byte boundaries.
- 32-bit** signed and unsigned words; these are aligned on

ARM Instruction Set Architecture - Features

- All ARM instructions are 32 bits wide.
- The Load-store architecture.
- 3-Address data processing instructions (that is, the two source operand registers and the result register are all independently specified).
- Conditional execution of every instruction.
- Inclusion of very powerful load and store multiple register instructions.
- Ability to perform a general shift operation and

Load-store architecture

- The instruction set will only process [ADD, SUB] values which are in registers [Immediate data] and will always place the results of such processing into a register.
 - **LOAD** instructions operations which apply to memory state are ones which copy memory values into registers.
 - **STORE** instructions copy register values into memory.
- **ARM** does not support '**memory-to-memory**' operations.
- ARM instructions fall into one of the following **3 categories**.
 - **Data processing** instructions.
 - Ex: **MOV, ADD, SUB, CMP**, etc.,
 - **Data transfer** instructions.
 - Ex: **LDR, STR**
 - **Control flow** instructions
 - Ex: **B, BGT, BEQ, BL**, etc.,

The RISC Features rejected by ARM

• Register windows.

- RISC processors incorporated a large number of registers.
- Procedure entry and exit instructions moved the visible 'window' to give each procedure access to new registers.
- Thereby reducing the data traffic between the processor and memory resulting from register saving and restoring.
- The principal problem with register windows is the large chip area occupied by the large number of registers.

The RISC Features

• **rejected by ARM** **Delayed branches.**

- Branches cause pipeline problems since they **interrupt** the smooth flow of instructions.
- RISC processors **ameliorate** the problem by using delayed branches where the branch takes effect *after* the following instruction has executed.
- ~~Work well~~ on single issue pipelined processors

But, **do not scale well** to super-scalar

implementations &

can interact **badly** with branch prediction

mechanisms.

• **Single-cycle execution of all instructions.**

- ARM **executes** most **data processing instructions** in a single clock cycle, **many** other instructions take **multiple clock cycles**.
- A simple load or store instruction requires at least two memory accesses.
- Single cycle operation of all instructions is only possible with **separate** data and instruction memories.

1. Which category of instructions typically accounts for the largest proportion of execution time in ARM-based processors?

- A. Arithmetic and Logical instructions
- B. Data movement instructions
- C. Comparison instructions
- D. Floating-point instructions

2. Which sequence correctly represents the stages of the ARM 3-stage pipeline?

- A. Fetch → Decode → Execute
- B. Fetch → Execute → Write Back
- C. Decode → Execute → Memory Access
- D. Fetch → Memory Access → Execute

3. Why does the ARM architecture not support memory-to-memory arithmetic operations?

- A. To simplify compiler design
- B. To reduce instruction length
- C. To enforce a load-store architecture
- D. To improve branch prediction accuracy

4. What is the primary purpose of the Saved Program Status Register (SPSR) in the ARM architecture?

- A. To store arithmetic and logical results
- B. To hold the return address of subroutines
- C. To preserve processor state during mode switching
- D. To control memory access permissions

1. Which category of instructions typically accounts for the largest proportion of execution time in ARM-based processors?

- A. Arithmetic and logical instructions
- B. Data movement instructions
- C. Comparison instructions
- D. Floating-point instructions

Answer - Data movement instructions

2. Which sequence correctly represents the stages of the ARM 3-stage pipeline?

- A. Fetch → Decode → Execute
- B. Fetch → Execute → Write Back
- C. Decode → Execute → Memory Access
- D. Fetch → Memory Access → Execute

Answer - Fetch → Decode → Execute

3. Why does the ARM architecture not support memory-to-memory arithmetic operations?

- A. To simplify compiler design
- B. To reduce instruction length
- C. To enforce a load-store architecture
- D. To improve branch prediction accuracy

Answer - To enforce a load-store architecture

4. What is the primary purpose of the Saved Program Status Register (SPSR) in the ARM architecture?

- A. To store arithmetic and logical results
- B. To hold the return address of subroutines
- C. To preserve processor state during mode switching
- D. To control memory access permissions

Answer - To preserve processor state mode switching



THANK YOU



V R BADRI PRASAD

badri.prasad@pes.edu

Department of Computer Science and
Engineering